

Resolução do Exame Modelo de Recurso — Bases de Dados (Modelo 2)

 **Ano Letivo:** 2025/2026 |  **Época:** Recurso (Modelo)

 **Curso:** Engenharia Informática / Segurança Informática em Redes de Computadores

 **Instituição:** P.PORTO — ESTG

 **Unidade Curricular:** Bases de Dados

1. Arquitetura ANSI/SPARC (2 val.)

? Pergunta 1: A arquitetura ANSI/SPARC identifica três níveis nos SGBD. Descreva pormenorizadamente o nível intermédio, identificando o seu nome, e o que se pretende que este nível represente. Explique de que forma este nível contribui para a independência de dados.

 **Resposta:** A arquitetura ANSI/SPARC divide a base de dados em três níveis de abstração: **Nível Externo** (visões individuais dos utilizadores), **Nível Conceptual** (esquema lógico global) e **Nível Interno** (armazenamento físico em disco).

O nível intermédio designa-se **Nível Conceptual** e representa a **visão lógica e global de toda a base de dados** para toda a organização. É neste nível que o Administrador da Base de Dados (DBA) desenha o esquema conceptual.

O que este nível representa:

- A definição de todas as **tabelas** (entidades), **colunas** (atributos) e os **relacionamentos** existentes entre elas.
- As **regras e restrições de integridade** impostas (chaves primárias, chaves estrangeiras, restrições CHECK e UNIQUE).
- As regras de **acesso, autorização e segurança** lógicas globais da base de dados.

Contribuição para a independência de dados: O nível conceptual funciona como camada de isolamento entre o nível interno (físico) e o nível externo (aplicações). Garante a **independência física** ao permitir que alterações no armazenamento em disco (reorganizar ficheiros, criar índices, mudar partições) sejam mapeadas internamente sem afetar o esquema lógico. E suporta a **independência lógica** ao permitir que o esquema conceptual evolua (adicionar colunas, dividir tabelas) enquanto as vistas do nível externo continuam a simular as estruturas originais para as aplicações.

2. Arquitetura Cliente-Servidor (2 val.)

? **Pergunta 2:** Compare a arquitetura cliente-servidor de dois níveis com a de três níveis e identifique, justificando, qual a mais adequada para a Web.

 **Resposta:**

Arquitetura de 2 níveis (2-tier): A aplicação cliente (fat client) comunica diretamente com o servidor de bases de dados. A máquina cliente é responsável por renderizar a interface gráfica e processar todas as regras de negócio. O servidor de dados apenas executa as queries SQL solicitadas.

Arquitetura de 3 níveis (3-tier): Introduce-se uma camada intermédia dedicada — o **Servidor de Aplicação** (Application Server). O cliente corre uma interface leve (thin client / browser). O Servidor de Aplicação processa toda a lógica e regras de negócio, e comunica com o Servidor de Base de Dados para leitura e gravação dos dados físicos.

Adequabilidade para a Web — a arquitetura de 3 níveis é a mais adequada porque:

1. **Escalabilidade e Pooling de Conexões:** O servidor de aplicação mantém um pool fechado de conexões abertas com a BD, reutilizando-as de forma concorrente para servir milhares de pedidos web em simultâneo. Numa arquitetura de 2 níveis, cada browser precisaria de abrir uma conexão direta ao SGBD, esgotando os limites de conexão rapidamente.
2. **Facilidade de Manutenção:** Atualizações à lógica de negócio são feitas exclusivamente no servidor central de aplicação, sem necessidade de redistribuir software pelos terminais dos utilizadores.
3. **Segurança:** O cliente web não tem acesso às credenciais de administrador nem acesso direto à base de dados, prevenindo ataques maliciosos à integridade dos dados.

3. **Cursors SQL (2 val.)**

? **Pergunta 3:** O que são cursores SQL? Qual o propósito da sua utilização? Descreva o ciclo de vida típico de um cursor, identificando cada uma das suas fases.

 **Resposta:** Um **cursor** é uma estrutura de controlo mantida pelo SGBD que funciona como um apontador lógico e permite às linguagens procedimentais iterar e processar os registos resultantes de uma consulta SELECT **linha a linha** (abordagem procedimental *one-record-at-a-time*), contrariando a natureza declarativa natural do SQL que atua em conjuntos de dados (*set-at-a-time*).

Propósito da utilização: Servem para desenvolver algoritmos de manipulação complexos, validações individuais registo a registo e processamento sequencial de tuplos em blocos de código procedimental (PL/SQL, T-SQL), onde a lógica requer tratar cada linha isoladamente.

Ciclo de vida do cursor:

Fase	Comando	Descrição
1. Declaração	DECLARE	Define a query SELECT associada ao cursor e atribui-lhe um nome.
2. Abertura	OPEN	Executa a query SELECT, cria o conjunto de resultados e aloca os recursos de memória.
3. Obtenção	FETCH	Obtém a linha corrente do resultado, copia os dados para variáveis e avança o apontador para a linha seguinte. Tipicamente inserido num ciclo (WHILE/LOOP).
4. Fecho	CLOSE	Fecha o cursor, invalida o conjunto de resultados e liberta os locks ativos sobre os dados.
5. Desalocação	DEALLOCATE	Remove a definição do cursor da memória, libertando completamente os recursos alocados.

4. LMD Procedimentais vs Não-Procedimentais (2 val.)

? **Pergunta 4:** Explique as diferenças existentes entre LMD procedimentais e não-procedimentais. Dê exemplos de linguagens que conheça para cada tipo.

 **Resposta:**

LMD Procedimentais:

- O utilizador especifica **como** os dados devem ser obtidos, definindo a sequência de passos físicos e a lógica algorítmica de acesso.
- Exigem a definição de um algoritmo com controlo de fluxo (laços, condicionais).
- Operam **registo a registo** (*one-record-at-a-time*).
- **Exemplos:** Álgebra Relacional, blocos de código procedimental com cursores em PL/SQL (Oracle) e T-SQL (SQL Server).

LMD Não-Procedimentais (Declarativas):

- O utilizador especifica apenas **o que** quer obter, sem detalhar o caminho físico ou os passos necessários.

- O **otimizador do SGBD** encarrega-se de definir o plano físico de execução mais eficiente.
- Operam em **conjuntos de dados** (*set-at-a-time*).
- **Exemplos:** Instrução `SELECT` em SQL, Cálculo Relacional (de tuplos e de domínios).

Resumo comparativo:

Característica	Procedimental	Não-Procedimental
Especifica	Como obter	O que obter
Processamento	Registo a registo	Conjunto de dados
Otimização	Manual (pelo programador)	Automática (pelo SGBD)
Exemplos	Álgebra Relacional, Cursores	SQL SELECT, Cálculo Relacional

5. Normalização: Objetivos e Impacto no Desempenho (2 val.)

? Pergunta 5: *Quais os objetivos da normalização de dados? De que forma o processo de normalização poderá afetar, posteriormente, o desempenho da respetiva implementação?*

 Resposta: No modelo relacional, a **normalização** visa decompor relações complexas em esquemas mais simples com base nas suas chaves e dependências funcionais, de forma a:

1. **Minimizar a redundância de dados** — evitar a duplicação desnecessária de informação.
2. **Eliminar anomalias de atualização** — garantir consistência em operações de inserção, remoção e modificação.
3. **Garantir a integridade e consistência lógica** — as dependências funcionais mantêm-se corretamente preservadas.
4. **Organizar os dados logicamente** — o modelo relacional fica mais limpo, legível e escalável.

Impacto no desempenho:

- **Nas operações de leitura/consulta (OLAP):** O desempenho pode ser **prejudicado**. Como os dados ficam distribuídos por várias tabelas menores, as consultas exigem a realização de múltiplas junções (**JOIN**), aumentando o processamento de CPU e o número de acessos de E/S (leitura em disco).
- **Nas operações de escrita/atualização (OLTP):** O desempenho é **otimizado**. Como as tabelas são mais estreitas e não há redundância, as escritas ocorrem num único registo de forma mais rápida e segura, sem necessidade de atualizar réplicas em múltiplos locais.

Nota sobre Desnormalização: É o processo intencional de reverter parcialmente a normalização, introduzindo alguma redundância controlada, com o objetivo de melhorar o desempenho de leitura em cenários específicos (ex: blogs, dashboards analíticos).

6. Atributos no Modelo Entidade-Relacionamento (2 val.)

? **Pergunta 6:** Descreva o que representam os atributos num diagrama ER e dê exemplos de atributos simples, compostos, multi-valor e derivados. Identifique a representação gráfica de cada tipo.

 **Resposta:** Num diagrama Entidade-Relacionamento (ER), os **atributos** representam as propriedades ou características individuais que descrevem uma entidade ou um relacionamento.

Classificação, Exemplos e Notação de Chen:

Tipo	Definição	Exemplo	Representação Gráfica
Simples (Atómico)	Contém um valor único e indivisível	NIF, Código do Produto	Elipse simples ligada à entidade
Composto	Pode ser decomposto em subatributos independentes	Morada (Rua + Localidade + Código Postal)	Elipse principal ligada a elipses secundárias
Multi-valor	Admite mais do que um valor para a mesma entidade	Telefone (vários contactos), Hobbies	Elipse de contorno duplo (dois círculos concêntricos)
Derivado	Calculado a partir de outros atributos existentes	Idade (calculada a partir da Data de Nascimento)	Elipse com contorno tracejado

Exemplo prático integrado: Na entidade Pessoa, o NIF é um atributo **simples**, a Morada é um atributo **composto** (decomposto em Rua, Localidade e Código Postal), os Telefones são um atributo **multi-valor** (uma pessoa pode ter vários números de contacto), e a Idade é um atributo **derivado** (calculada automaticamente a partir da Data de Nascimento e da data atual).

7. Exercício de Normalização de Contrato (3 val.)

? *Pergunta 7: Normalização do contrato de aluguer da AutoFlex Rent-a-Car.*

 Resposta:

Identificação dos Atributos

Letra	Atributo
A	NIF_Empresa
B	Nome_Empresa
C	Morada_Empresa
D	NumContrato
E	DataInicio
F	CodAgenciaLev
G	NomeAgenciaLev
H	CodAgenciaDev
I	NomeAgenciaDev
J	NIF_Condutor

Letra	Atributo
K	Nome_Conductor
L	CartaConducao
M	CategoriaCC
N	Matricula
O	Marca
P	Modelo
Q	CategoriaViat
R	PrecoDiario
S	Combustivel
T	CodExtra
U	DescricaoExtra
V	PrecoExtraDia
W	Duracao
X	TotalFatura
Y	MetodoPagamento

Letra	Atributo
Z	DataDevolucao

Forma Não Normalizada (FNN)

Definição: Uma tabela que contém um ou mais grupos repetidos.

Todos os atributos residem numa única relação com dois grupos repetitivos (condutores e extras):

```
Contrato_FNN(D, A, B, C, E, F, G, H, I, N, O, P, Q, R, S, W, X, Y, Z,
              [J, K, L, M],           ← grupo repetido: condutores
              [T, U, V])             ← grupo repetido: extras
```

1 Primeira Forma Normal (1FN)

Definição: Uma relação em que a intersecção entre uma linha e uma coluna contém um e um só valor (valores atômicos). Não deve conter grupos repetidos.

Achatando os grupos repetitivos, a PK passa a ser composta:

```
Contrato_1FN(NumContrato, NIF_Condutor, CodExtra,
              NIF_Empresa, Nome_Empresa, Morada_Empresa,
              DataInicio, CodAgenciaLev, NomeAgenciaLev,
              CodAgenciaDev, NomeAgenciaDev,
              Nome_Condutor, CartaConducao, CategoriaCC,
              Matricula, Marca, Modelo, CategoriaViat,
              PrecoDiario, Combustivel,
              DescricaoExtra, PrecoExtraDia,
              Duracao, TotalFatura, MetodoPagamento, DataDevolucao)
PK: (NumContrato, NIF_Condutor, CodExtra)
```

Dependências Funcionais (DFs) verificadas:

- \$NumContrato \rightarrow NIF_Empresa, Nome_Empresa, Morada_Empresa, DataInicio, CodAgenciaLev, NomeAgenciaLev, CodAgenciaDev, NomeAgenciaDev, Matricula, Marca, Modelo, CategoriaViat, PrecoDiario, Combustivel, Duracao, TotalFatura, MetodoPagamento, DataDevolucao\$
- \$NIF_Condutor \rightarrow Nome_Condutor, CartaConducao, CategoriaCC\$
- \$CodExtra \rightarrow DescricaoExtra, PrecoExtraDia\$
- \$NIF_Empresa \rightarrow Nome_Empresa, Morada_Empresa\$

- $\$CodAgenciaLev \rightarrow NomeAgenciaLev\$$
- $\$CodAgenciaDev \rightarrow NomeAgenciaDev\$$
- $\$Matricula \rightarrow Marca, Modelo, CategoriaViat, Combustivel\$$

2 Segunda Forma Normal (2FN)

Definição: Uma relação que está na 1FN e todos os atributos não primos dependem totalmente da chave primária (sem dependências parciais).

Decompondo as dependências parciais sobre a chave composta (NumContrato, NIF_Condutor, CodExtra):

```
Cabecalho_2FN(NumContrato, NIF_Empresa, Nome_Empresa, Morada_Empresa,
               DataInicio, CodAgenciaLev, NomeAgenciaLev,
               CodAgenciaDev, NomeAgenciaDev,
               Matricula, Marca, Modelo, CategoriaViat,
               PrecoDiario, Combustivel,
               Duracao, TotalFatura, MetodoPagamento, DataDevolucao)
```

PK: NumContrato

```
Condutor_2FN(NIF_Condutor, Nome_Condutor, CartaConducao, CategoriaCC)
```

PK: NIF_Condutor

```
Extra_2FN(CodExtra, DescricaoExtra, PrecoExtraDia)
```

PK: CodExtra

```
ContratoCondutor_2FN(NumContrato, NIF_Condutor)
```

PK: (NumContrato, NIF_Condutor)

```
ContratoExtra_2FN(NumContrato, CodExtra)
```

PK: (NumContrato, CodExtra)

3 Terceira Forma Normal (3FN)

Definição: Uma relação que está na 2FN e na qual nenhum atributo não pertencente à chave primária depende transitivamente de qualquer chave candidata.

Dependências transitivas detetadas em Cabecalho_2FN:

- $\$NIF_Empresa \rightarrow Nome_Empresa, Morada_Empresa\$$ (transitiva via NumContrato)
- $\$CodAgenciaLev \rightarrow NomeAgenciaLev\$$ (transitiva via NumContrato)
- $\$CodAgenciaDev \rightarrow NomeAgenciaDev\$$ (transitiva via NumContrato — mesma tabela Agência)

- \$Matricula \rightarrow Marca, Modelo, CategoriaViat, Combustivel\$ (transitiva via NumContrato)

Extraíndo estas dependências, obtemos o **esquema final normalizado (3FN)**:

```

Empresa(NIF_Empresa, Nome_Empresa, Morada_Empresa)
    PK: NIF_Empresa

Agencia(CodAgencia, NomeAgencia)
    PK: CodAgencia

Viatura(Matricula, Marca, Modelo, CategoriaViat, PrecoDiario, Combustivel)
    PK: Matricula

Condutor(NIF_Condutor, Nome_Condutor, CartaConducao, CategoriaCC)
    PK: NIF_Condutor

Extra(CodExtra, DescricaoExtra, PrecoExtraDia)
    PK: CodExtra

Contrato(NumContrato, DataInicio, NIF_Empresa, CodAgenciaLev,
        CodAgenciaDev, Matricula, Duracao, TotalFatura,
        MetodoPagamento, DataDevolucao)
    PK: NumContrato
    FK: NIF_Empresa → Empresa(NIF_Empresa)
    FK: CodAgenciaLev → Agencia(CodAgencia)
    FK: CodAgenciaDev → Agencia(CodAgencia)
    FK: Matricula → Viatura(Matricula)

ContratoCondutor(NumContrato, NIF_Condutor)
    PK: (NumContrato, NIF_Condutor)
    FK: NumContrato → Contrato(NumContrato)
    FK: NIF_Condutor → Condutor(NIF_Condutor)

ContratoExtra(NumContrato, CodExtra)
    PK: (NumContrato, CodExtra)
    FK: NumContrato → Contrato(NumContrato)
    FK: CodExtra → Extra(CodExtra)

```

Nota: As agências de levantamento e devolução referenciam a mesma tabela `Agencia`, pois partilham a mesma estrutura (código + nome). O atributo `PrecoDiario` foi movido

para *Viatura* pois depende funcionalmente da matrícula (cada viatura tem um preço diário definido pela sua categoria).

8. Modelação, SQL e Álgebra Relacional (5 val.)

a) Chave primária e chaves estrangeiras da tabela *Inscricao* (1 val.)

? **Pergunta 8a:** Identifique a PK e FKs da tabela *Inscricao*. Justifique a escolha da PK.

 **Resposta:**

```
Inscricao(numSocio, codAula, dataInscricao, presenca)
```

```
PK: (numSocio, codAula)
```

```
FK: numSocio → Socio(numSocio)
```

```
FK: codAula → Aula(codAula)
```

Justificação: A chave primária é composta por (numSocio, codAula) porque identifica univocamente cada inscrição — um sócio inscreve-se no máximo uma vez em cada aula. As chaves estrangeiras são numSocio, que referencia o sócio na tabela Socio, e codAula, que referencia a aula na tabela Aula. Estas FKs garantem a integridade referencial — não é possível criar uma inscrição para um sócio ou aula inexistente.

b) SQL: Instrutores com mais de 3 aulas diferentes com pelo menos 20 inscrições cada (2 val.)

? **Pergunta 8b (SQL):** Quais os instrutores que dão mais de 3 aulas diferentes com pelo menos 20 inscrições cada?

 **Resposta:**

Primeiro, identificamos as aulas que têm pelo menos 20 inscrições. Depois, juntamos com a tabela *Instrutor* e contamos quantas dessas aulas cada instrutor leciona. Finalmente, filtramos os instrutores com mais de 3 aulas que cumpram o critério:

```
SELECT I.codInst, I.nome
FROM Instrutor I
INNER JOIN Aula A ON I.codInst = A.codInst
INNER JOIN Inscricao INS ON A.codAula = INS.codAula
GROUP BY I.codInst, I.nome, A.codAula
HAVING COUNT(*) >= 20
```

A query acima dá as aulas com ≥ 20 inscrições por instrutor. Para obter os instrutores com mais de 3 dessas aulas, usamos uma subquery:

```
SELECT codInst, nome
FROM (
    SELECT I.codInst, I.nome, A.codAula
    FROM Instrutor I
    INNER JOIN Aula A ON I.codInst = A.codInst
    INNER JOIN Inscricao INS ON A.codAula = INS.codAula
    GROUP BY I.codInst, I.nome, A.codAula
    HAVING COUNT(*) >= 20
) AS AulasPopulares
GROUP BY codInst, nome
HAVING COUNT(*) > 3;
```

Alternativa com subquery no HAVING:

```
SELECT I.codInst, I.nome
FROM Instrutor I
WHERE (
    SELECT COUNT(*)
    FROM Aula A
    WHERE A.codInst = I.codInst
    AND (SELECT COUNT(*) FROM Inscricao INS WHERE INS.codAula = A.codAula) >=
20
) > 3;
```



c) Álgebra Relacional: Sócios VIP que nunca se inscreveram em Spinning (2 val.)

? Pergunta 8c (Álgebra Relacional): Quais os sócios com plano VIP que nunca se inscreveram em nenhuma aula de Spinning?



Resposta:

Selecionamos os sócios VIP e projetamos os seus números. Depois, encontramos as aulas de Spinning, juntamos com as inscrições para encontrar os sócios que frequentaram Spinning, e subtraímos este conjunto dos sócios VIP:

$$\text{\$}\$ \text{SociosVIP} \rightarrow \pi_{\text{\{numSocio\}}}(\sigma_{\text{\{plano = 'VIP'\}}}(\text{Socio}))\text{\$}\$$$
$$\text{\$}\$ \text{AulasSpinning} \rightarrow \sigma_{\text{\{modalidade = 'Spinning'\}}}(\text{Aula})\text{\$}\$$$

\$\$SociosComSpinning \leftarrow \pi_{\text{numSocio}}(\text{Inscricao \bowtie AulasSpinning})\$\$

\$\$Resultado \leftarrow \text{SociosVIP} - \text{SociosComSpinning}\$\$